리눅스 정리 — DevOps 관점

> 작성일: 2026-04-01
> 대상: DevOps 입문~중급 학습자
> 범위: 리눅스 기초부터 파일시스템, 프로세스, systemd, 쉘 스크립팅, 컨테이너 기반 기술, 보안까지

목차

1. 리눅스 기초 — 왜 DevOps에서 필수인가
2. 리눅스 배포판 비교
3. 파일시스템 계층 구조 (FHS)
4. 파일 및 디렉터리 관리
5. 파일 권한과 소유권
6. 사용자와 그룹 관리
7. 프로세스 관리
8. systemd와 서비스 관리
9. 패키지 관리
10. 쉘과 Bash 스크립팅
11. 텍스트 처리 — grep, sed, awk
12. 디스크와 스토리지 관리
13. 로그 관리
14. 시스템 모니터링
15. SSH와 원격 접속
16. cron과 작업 스케줄링
17. 네트워크 설정과 진단
18. 컨테이너 기반 리눅스 기술
19. 보안 강화 (Hardening)
20. 트러블슈팅 가이드
21. 참고 자료

---

1. 리눅스 기초 — 왜 DevOps에서 필수인가

서버 인프라의 대부분은 리눅스 위에서 동작한다. AWS EC2, Docker 컨테이너, Kubernetes 노드 모두 리눅스 커널 기반이다.

- CI/CD 러너 → 리눅스 환경에서 빌드/테스트 실행
- 컨테이너 런타임 → 리눅스 커널의 namespace, cgroup 활용
- 클라우드 인스턴스 → 90% 이상이 리눅스 기반
- 인프라 자동화 (Ansible, Terraform) → 리눅스 명령어 기반
- 장애 대응 → 리눅스 로그, 프로세스, 네트워크 진단 필수

리눅스를 모르면 서버 접속 후 아무것도 할 수 없다. DevOps에서 리눅스는 선택이 아니라 **기반 기술**이다.

리눅스 커널 아키텍처 개요

```
┌──────────────────────────────────────────────────────────┐
│                  사용자 공간 (User Space)                  │
│                                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────────────┐      │
│   │  Shell   │  │ 시스템   │  │  사용자          │  │
│   │ (bash,zsh)│  │ 데몬     │  │  애플리케이션    │  │
│   └──────────┘  └──────────┘  └──────────────────┘      │
│        │             │                │                  │
├────────┼─────────────┼────────────────┼──────────────────┤
│              시스템 콜 인터페이스 (System Call)            │
├──────────────────────────────────────────────────────────┤
│                  커널 공간 (Kernel Space)                  │
│                                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────────────┐      │
│   │ 프로세스 │  │ 메모리   │  │  파일시스템      │  │
│   │ 관리     │  │ 관리     │  │  (ext4, xfs)     │  │
│   └──────────┘  └──────────┘  └──────────────────┘      │
│                                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────────────┐      │
│   │ 네트워크 │  │ 장치     │  │  보안 모듈       │  │
│   │ 스택     │  │ 드라이버 │  │  (SELinux,AppArmor)│  │
│   └──────────┘  └──────────┘  └──────────────────┘      │
├──────────────────────────────────────────────────────────┤
│                  하드웨어 (Hardware)                      │
│        CPU    메모리    디스크    NIC    GPU              │
└──────────────────────────────────────────────────────────┘
```

---

2. 리눅스 배포판 비교

DevOps에서 자주 사용하는 주요 배포판을 비교한다.

배포판	패키지 관리자	패키지 형식	기본 init	주요 사용처	DevOps 활용도
Ubuntu	apt	.deb	systemd	클라우드 서버, Docker 호스트	★★★★★
Debian	apt	.deb	systemd	안정성 중시 서버	★★★★☆
CentOS/Rocky	dnf (yum)	.rpm	systemd	엔터프라이즈 서버	★★★★☆
Amazon Linux	dnf (yum)	.rpm	systemd	AWS EC2 최적화	★★★★★
Alpine	apk	.apk	OpenRC	Docker 이미지 (경량)	★★★★★
Arch Linux	pacman	.pkg.tar	systemd	개발 환경, 최신 패키지	★★☆☆☆

배포판 선택 기준

```
Docker 이미지가 목적인가?
├── YES → Alpine (5MB 베이스 이미지, 보안에 유리)
```

```
└── NO
    ├── AWS 환경인가?
    │   ├── YES → Amazon Linux 2023 (AWS 서비스 최적화)
    │   └── NO
    │       ├── 엔터프라이즈/장기 지원 필요?
    │       │   ├── YES → Rocky Linux / Ubuntu LTS
    │       │   └── NO → Ubuntu (커뮤니티 크고 자료 풍부)
    │       └──
    └──
```

> **DevOps 팁**: 운영 환경의 배포판을 통일하면 패키지 관리, 자동화 스크립트, 트러블슈팅이 모두 단순해진다.

---

3. 파일시스템 계층 구조 (FHS)

리눅스는 **FHS(Filesystem Hierarchy Standard)** 에 따라 디렉터리 구조가 표준화되어 있다. 어떤 배포판이든 기본 구조는 동일하다.

```
/                          ← 루트 디렉터리 (모든 것의 시작점)
├── /bin                   ← 필수 바이너리 (ls, cp, cat, bash)
├── /sbin                  ← 시스템 관리 바이너리 (fdisk, iptables)
├── /etc                   ← 시스템 설정 파일 (nginx.conf, fstab)
├── /home                  ← 일반 사용자 홈 디렉터리
│   └── /home/ubuntu       ← ubuntu 사용자의 홈
├── /root                  ← root 사용자의 홈 디렉터리
├── /var                   ← 가변 데이터 (로그, 캐시, 메일)
│   ├── /var/log           ← 시스템 로그
│   ├── /var/lib           ← 애플리케이션 상태 데이터
│   └── /var/cache         ← 캐시 데이터
├── /tmp                   ← 임시 파일 (재부팅 시 삭제)
├── /usr                   ← 사용자 프로그램과 데이터
│   ├── /usr/bin           ← 일반 사용자 명령어
│   ├── /usr/sbin          ← 시스템 관리자 명령어
│   ├── /usr/lib           ← 라이브러리
│   └── /usr/local         ← 수동 설치 프로그램
├── /opt                   ← 서드파티 소프트웨어
├── /dev                   ← 디바이스 파일
├── /proc                  ← 프로세스/커널 정보 (가상 파일시스템)
├── /sys                   ← 커널/하드웨어 정보 (가상 파일시스템)
├── /boot                  ← 부트로더, 커널 이미지
├── /mnt                   ← 임시 마운트 포인트
├── /media                 ← 이동식 미디어 자동 마운트
└── /srv                   ← 서비스 관련 데이터
```

DevOps에서 자주 접근하는 경로

경로	용도	예시
`/etc/nginx/`	Nginx 설정	`nginx.conf`, `sites-available/`

경로	용도	예시
`/etc/systemd/system/`	커스텀 서비스 유닛 파일	`myapp.service`
`/var/log/`	시스템 및 애플리케이션 로그	`syslog`, `auth.log`, `nginx/`
`/var/lib/docker/`	Docker 데이터	이미지, 컨테이너, 볼륨
`/proc/cpuinfo`	CPU 정보 확인	`cat /proc/cpuinfo`
`/proc/meminfo`	메모리 정보 확인	`cat /proc/meminfo`
`/etc/ssh/sshd_config`	SSH 서버 설정	포트 변경, 키 인증 설정
`/etc/crontab`	시스템 크론 설정	주기적 작업 스케줄
`/etc/hosts`	로컬 DNS 매핑	IP ↔ 호스트명 매핑
`/etc/resolv.conf`	DNS 서버 설정	`nameserver 8.8.8.8`

---

4. 파일 및 디렉터리 관리

기본 명령어 정리

명령어	기능	사용 예시
`ls`	파일/디렉터리 목록	`ls -alh` (전체, 상세, 사람이 읽기 쉬운 크기)
`cd`	디렉터리 이동	`cd /var/log`
`pwd`	현재 경로 출력	`pwd`
`mkdir`	디렉터리 생성	`mkdir -p /app/config/env` (중간 경로 자동)
`rm`	파일/디렉터리 삭제	`rm -rf /tmp/build` (재귀 강제 삭제)
`cp`	복사	`cp -r src/ dest/` (디렉터리 재귀 복사)
`mv`	이동/이름 변경	`mv old.conf new.conf`
`touch`	빈 파일 생성/시간 갱신	`touch deploy.lock`
`find`	파일 검색	`find / -name "*.log" -mtime +7`
`ln`	링크 생성	`ln -s /etc/nginx/sites-available/app sites-enabled/app`
`stat`	파일 상세 정보	`stat /etc/passwd`
`file`	파일 타입 확인	`file script.sh`

하드 링크 vs 심볼릭 링크

```
[하드 링크]                                [심볼릭 링크 (소프트 링크)]

 파일 A ──→ inode 123 ←── 하드 링크 B       파일 A ──→ inode 123
                                            심볼릭 링크 B ──→ "파일 A" (경로 참조)
```

```
• 같은 inode를 가리킴                    • 다른 inode, 경로 문자열 저장
• 원본 삭제해도 데이터 유지                 • 원본 삭제 시 깨진 링크 (dangling)
• 같은 파일시스템 내에서만 가능              • 파일시스템 간 가능, 디렉터리도 가능
• ln 파일A 하드링크B                      • ln -s 파일A 심볼릭링크B
```

> **DevOps 팁**: Nginx, Apache 등의 설정에서 `sites-available` → `sites-enabled` 연결은 심볼릭 링크를 사용한다. 배포 자동화 시 `ln -sf`로 설정 전환이 가능하다.

---

5. 파일 권한과 소유권

리눅스의 파일 권한은 **보안의 기본**이다. 잘못된 권한 설정은 보안 사고 또는 서비스 장애로 직결된다.

권한 구조

```
-rwxr-xr-- 1 ubuntu devops 4096 Apr 01 12:00 deploy.sh
│├┤├┤├┤   │      │
│ │  │  │    │      └── 소유 그룹
│ │  │  │    └── 소유자
│ │  │  └── 기타 사용자 (Others): r-- (읽기만 가능)
│ │  └── 소유 그룹 (Group): r-x (읽기, 실행)
│ └── 소유자 (Owner): rwx (읽기, 쓰기, 실행)
└── 파일 타입: - (일반 파일), d (디렉터리), l (심볼릭 링크)
```

권한 숫자 표기법 (Octal)

권한	숫자	의미
`r--`	4	읽기 (Read)
`-w-`	2	쓰기 (Write)
`--x`	1	실행 (Execute)
`rwx`	7	읽기+쓰기+실행
`r-x`	5	읽기+실행
`rw-`	6	읽기+쓰기
`r--`	4	읽기만
`---`	0	권한 없음

자주 사용하는 권한 설정

숫자	의미	대표 사용처
755	`rwxr-xr-x`	실행 파일, 디렉터리

숫자	의미	대표 사용처
644	`rw-r--r--`	일반 설정 파일
600	`rw-------`	SSH 키 파일 (`~/.ssh/id_rsa`)
700	`rwx------`	`.ssh` 디렉터리
400	`r--------`	AWS 키 페어 `.pem` 파일
777	`rwxrwxrwx`	절대 사용 금지 (보안 취약)

권한 관련 명령어

```
# 권한 변경
chmod 755 deploy.sh          # 숫자 표기법
chmod u+x script.sh          # 소유자에게 실행 권한 추가
chmod -R 644 /etc/nginx/     # 재귀적 권한 변경

# 소유권 변경
chown ubuntu:devops app/     # 소유자:그룹 변경
chown -R www-data:www-data /var/www/   # 웹 서버 디렉터리

# 특수 권한
chmod u+s /usr/bin/passwd    # SUID: 실행 시 소유자 권한으로 실행
chmod g+s /shared/           # SGID: 생성되는 파일이 디렉터리의 그룹 상속
chmod +t /tmp/               # Sticky Bit: 소유자만 삭제 가능
```

특수 권한 정리

특수 권한	숫자	표기	의미	예시
SUID	4000	`-rws`	실행 시 파일 소유자의 권한으로 실행	`/usr/bin/passwd`
SGID	2000	`-rwxs`	실행 시 파일 그룹의 권한으로 실행	공유 디렉터리
Sticky Bit	1000	`drwxt`	디렉터리 내 파일을 소유자만 삭제 가능	`/tmp`

> **DevOps 팁**: CI/CD에서 빌드 아티팩트의 권한이 잘못되면 배포가 실패한다. Dockerfile에서 `COPY --chown=` 옵션이나 `RUN chmod`로 적절한 권한을 설정하자.

---

6. 사용자와 그룹 관리

사용자 관련 주요 파일

파일	내용	예시
`/etc/passwd`	사용자 계정 정보	`ubuntu:x:1000:1000::/home/ubuntu:/bin/bash`
`/etc/shadow`	암호화된 비밀번호	`ubuntu:$6$...:19000:0:99999:7:::`

파일	내용	예시
`/etc/group`	그룹 정보	`devops:x:1001:ubuntu,deploy`
`/etc/sudoers`	sudo 권한 설정	`ubuntu ALL=(ALL) NOPASSWD:ALL`

`/etc/passwd` 구조

```
ubuntu : x : 1000 : 1000 : Ubuntu User : /home/ubuntu : /bin/bash
  │      │    │       │        │              │             │
  │      │    │       │        │              │             └── 로그인 쉘
  │      │    │       │        │              └── 홈 디렉터리
  │      │    │       │        └── 설명 (GECOS)
  │      │    │       └── 기본 그룹 GID
  │      │    └── 사용자 UID
  │      └── 비밀번호 (x = /etc/shadow 참조)
  └── 사용자 이름
```

사용자/그룹 관리 명령어

```
# 사용자 생성
useradd -m -s /bin/bash -G docker,sudo deploy
#        │   │           │
#        │   │           └── 추가 그룹
#        │   └── 로그인 쉘
#        └── 홈 디렉터리 자동 생성

# 비밀번호 설정
passwd deploy

# 사용자 수정
usermod -aG docker ubuntu   # ubuntu를 docker 그룹에 추가
usermod -s /sbin/nologin svc_account  # 서비스 계정 로그인 차단

# 사용자 삭제
userdel -r olduser          # 홈 디렉터리까지 삭제

# 그룹 관리
groupadd devops             # 그룹 생성
groupdel devops             # 그룹 삭제

# 확인
id ubuntu                   # uid, gid, 그룹 목록 확인
groups ubuntu               # 소속 그룹 확인
whoami                      # 현재 사용자 확인
```

sudo 설정

```
# /etc/sudoers (반드시 visudo 명령으로 편집)

# 사용자별 설정
deploy  ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx
#  │      │   │      │           │
#  │      │   │      │           └── 허용할 명령어
#  │      │   │      └── 비밀번호 없이
#  │      │   └── 모든 사용자로 실행 가능
#  │      └── 모든 호스트에서
#  └── 대상 사용자

# 그룹별 설정
%devops ALL=(ALL) NOPASSWD:ALL   # devops 그룹 전체에 sudo 권한
```

> **DevOps 팁**: 운영 서버에서는 root 직접 로그인을 비활성화하고, 서비스 전용 계정은 `/sbin/nologin` 쉘을 지정하여 로그인을 차단한다. Ansible 등 자동화 도구는 전용 사용자로 실행한다.

---

7. 프로세스 관리

프로세스란?

실행 중인 프로그램의 인스턴스다. 모든 프로세스는 고유한 **PID (Process ID)** 를 가진다.

```
                    ┌─────────┐
                    │ PID 1   │
                    │ systemd │  ← 모든 프로세스의 부모
                    └─────────┘
                         │
          ┌──────────────┼──────────────┐
          v              v              v
     ┌─────────┐    ┌─────────┐    ┌──────────┐
     │ PID 500 │    │ PID 600 │    │ PID 700  │
     │ sshd    │    │ nginx   │    │ dockerd  │
     └─────────┘    └─────────┘    └──────────┘
          │              │              │
          v              v              v
     ┌─────────┐    ┌─────────┐    ┌──────────┐
     │ PID 1200│    │ PID 1300│    │ PID 1400 │
     │ bash    │    │ worker  │    │ containerd│
     └─────────┘    └─────────┘    └──────────┘
```

프로세스 상태

상태	코드	설명
실행	R	CPU에서 실행 중 또는 실행 대기
수면	S	이벤트 대기 중 (인터럽트 가능)
수면	D	I/O 대기 중 (인터럽트 불가)

상태	코드	설명
정지	T	시그널에 의해 정지됨
좀비	Z	종료되었지만 부모가 아직 회수하지 않은 상태

프로세스 관리 명령어

```
# 프로세스 조회
ps aux                      # 모든 프로세스 목록 (BSD 형식)
ps -ef                      # 모든 프로세스 목록 (System V 형식)
ps aux | grep nginx         # 특정 프로세스 검색

# 실시간 모니터링
top                         # 실시간 프로세스 모니터
htop                        # top의 개선 버전 (색상, 트리 뷰)

# 프로세스 트리
pstree -p                   # 프로세스 트리 + PID 표시

# 프로세스 종료
kill PID                    # 기본 SIGTERM (정상 종료 요청)
kill -9 PID                 # SIGKILL (강제 종료)
kill -HUP PID               # SIGHUP (설정 리로드, Nginx 등)
killall nginx               # 이름으로 종료
pkill -f "java.*myapp"      # 패턴으로 종료

# 백그라운드 실행
./script.sh &               # 백그라운드 실행
nohup ./script.sh &         # 터미널 종료 후에도 유지
jobs                        # 백그라운드 작업 목록
fg %1                       # 작업 1을 포그라운드로
bg %1                       # 작업 1을 백그라운드로 재개
```

주요 시그널

시그널	번호	동작	사용 예시
SIGHUP	1	설정 리로드	`kill -1 PID` (Nginx 리로드)
SIGINT	2	인터럽트 (Ctrl+C)	터미널에서 실행 중인 프로세스 중단
SIGKILL	9	강제 종료 (무시 불가)	`kill -9 PID` (응답 없는 프로세스)
SIGTERM	15	정상 종료 요청	`kill PID` (기본값)
SIGSTOP	19	정지 (무시 불가)	디버깅 시 프로세스 일시 정지
SIGCONT	18	정지된 프로세스 재개	`kill -CONT PID`

> **DevOps 팁**: 컨테이너 환경에서 `docker stop`은 SIGTERM을 먼저 보내고, 10초 후 SIGKILL을 보낸다. 애플리케이션이 Graceful Shutdown을 구현해야 데이터 손실을 방지할 수 있다.

8. systemd와 서비스 관리

systemd란?

리눅스의 **init 시스템이자 서비스 관리자**다. PID 1로 실행되며 시스템 부팅, 서비스 관리, 로그 수집을 담당한다. 현재 대부분의 주요 배포판이 systemd를 사용한다.

systemctl 주요 명령어

명령어	기능
`systemctl start nginx`	서비스 시작
`systemctl stop nginx`	서비스 중지
`systemctl restart nginx`	서비스 재시작
`systemctl reload nginx`	설정 리로드 (무중단)
`systemctl enable nginx`	부팅 시 자동 시작 등록
`systemctl disable nginx`	자동 시작 해제
`systemctl status nginx`	서비스 상태 확인
`systemctl is-active nginx`	실행 중인지 확인
`systemctl is-enabled nginx`	자동 시작 등록 여부 확인
`systemctl list-units --type=service`	모든 서비스 목록
`systemctl daemon-reload`	유닛 파일 변경 후 리로드

서비스 유닛 파일 작성

`/etc/systemd/system/myapp.service` 경로에 유닛 파일을 생성한다.

```
[Unit]
Description=My Application Service
Documentation=https://docs.myapp.com
After=network.target postgresql.service
Wants=postgresql.service

[Service]
Type=simple
User=appuser
Group=appgroup
WorkingDirectory=/opt/myapp
Environment=NODE_ENV=production
EnvironmentFile=/opt/myapp/.env
ExecStartPre=/opt/myapp/pre-start.sh
ExecStart=/usr/bin/node /opt/myapp/server.js
ExecReload=/bin/kill -HUP $MAINPID
```

```
Restart=on-failure
RestartSec=5
StartLimitBurst=3
StartLimitIntervalSec=60

# 보안 강화
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/opt/myapp/data /var/log/myapp

# 리소스 제한
MemoryMax=512M
CPUQuota=80%

[Install]
WantedBy=multi-user.target
```

유닛 파일 섹션 설명

섹션	항목	설명
`[Unit]`	`After`	지정한 서비스 이후에 시작
`[Unit]`	`Wants`	의존 서비스 (실패해도 시작)
`[Unit]`	`Requires`	필수 의존 서비스 (실패 시 함께 중지)
`[Service]`	`Type`	`simple`, `forking`, `oneshot`, `notify` 등
`[Service]`	`Restart`	`no`, `on-failure`, `always` 등
`[Service]`	`RestartSec`	재시작 대기 시간 (초)
`[Service]`	`ExecStart`	서비스 실행 명령어
`[Service]`	`ExecReload`	리로드 시 실행할 명령어
`[Install]`	`WantedBy`	어떤 타겟에서 활성화할지 (multi-user.target 등)

서비스 등록 워크플로우

```
# 1. 유닛 파일 작성
sudo vi /etc/systemd/system/myapp.service

# 2. systemd에 새 유닛 파일 인식시키기
sudo systemctl daemon-reload

# 3. 서비스 시작
sudo systemctl start myapp

# 4. 상태 확인
sudo systemctl status myapp
```

```
# 5. 부팅 시 자동 시작 등록
sudo systemctl enable myapp
```

> **DevOps 팁**: `Restart=on-failure`와 `RestartSec=5`를 설정하면, 비정상 종료 시 5초 후 자동 재시작된다. 쿠버네티스의 restartPolicy와 유사한 개념이다.

---

9. 패키지 관리

Debian/Ubuntu 계열 (apt)

```
# 패키지 목록 갱신
sudo apt update

# 패키지 설치
sudo apt install -y nginx curl git

# 패키지 업그레이드
sudo apt upgrade -y              # 설치된 패키지 업그레이드
sudo apt full-upgrade -y         # 의존성 변경 포함 전체 업그레이드

# 패키지 제거
sudo apt remove nginx            # 패키지만 제거
sudo apt purge nginx             # 설정 파일까지 제거
sudo apt autoremove              # 불필요한 의존성 정리

# 패키지 검색/정보
apt search nginx
apt show nginx
dpkg -l | grep nginx             # 설치된 패키지 확인
dpkg -L nginx                    # 패키지의 파일 목록
```

RHEL/CentOS/Rocky 계열 (dnf/yum)

```
# 패키지 목록 갱신
sudo dnf check-update

# 패키지 설치
sudo dnf install -y nginx curl git

# 패키지 업그레이드
sudo dnf upgrade -y

# 패키지 제거
sudo dnf remove nginx
sudo dnf autoremove

# 패키지 검색/정보
```

```
dnf search nginx
dnf info nginx
rpm -qa | grep nginx            # 설치된 패키지 확인
rpm -ql nginx                   # 패키지의 파일 목록
```

Alpine (apk) — Docker 이미지용

```
# 패키지 설치
apk add --no-cache nginx curl

# 패키지 제거
apk del nginx

# 패키지 검색
apk search nginx
```

패키지 관리자 비교

작업	apt (Debian/Ubuntu)	dnf (RHEL/Rocky)	apk (Alpine)
목록 갱신	`apt update`	`dnf check-update`	`apk update`
설치	`apt install -y pkg`	`dnf install -y pkg`	`apk add pkg`
제거	`apt remove pkg`	`dnf remove pkg`	`apk del pkg`
전체 업그레이드	`apt upgrade -y`	`dnf upgrade -y`	`apk upgrade`
검색	`apt search keyword`	`dnf search keyword`	`apk search keyword`
정보 확인	`apt show pkg`	`dnf info pkg`	`apk info pkg`
파일 목록	`dpkg -L pkg`	`rpm -ql pkg`	`apk info -L pkg`

> **DevOps 팁**: Dockerfile에서 `apt install` 후 반드시 `rm -rf /var/lib/apt/lists/*`로 캐시를 삭제하여 이미지 크기를 줄인다. Alpine에서는 `apk add --no-cache`를 사용한다.

---

10. 쉘과 Bash 스크립팅

쉘 기본 개념

쉘은 사용자와 커널 사이의 **인터페이스**다. 명령어를 입력받아 커널에 전달하고 결과를 출력한다.

쉘	경로	특징
bash	`/bin/bash`	가장 널리 사용, 대부분의 배포판 기본 쉘
sh	`/bin/sh`	POSIX 호환 최소 쉘
zsh	`/bin/zsh`	bash 상위 호환, 자동완성 강력

쉘	경로	특징
dash	`/bin/dash`	Debian 기본 `/bin/sh`, 경량

Bash 스크립트 기본 구조

```
#!/bin/bash
# ==============================================================
# 스크립트: deploy.sh
# 설명: 애플리케이션 배포 자동화
# 사용법: ./deploy.sh [환경] [버전]
# ==============================================================

set -euo pipefail
# -e : 에러 발생 시 즉시 종료
# -u : 미정의 변수 사용 시 에러
# -o pipefail : 파이프라인 중간 에러도 감지

# ---- 변수 ----
ENV="${1:-staging}"
VERSION="${2:-latest}"
APP_DIR="/opt/myapp"
LOG_FILE="/var/log/deploy.log"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# ---- 함수 ----
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

rollback() {
    log "ERROR: 배포 실패 — 롤백 시작"
    ln -sfn "${APP_DIR}/releases/previous" "${APP_DIR}/current"
    systemctl restart myapp
    log "롤백 완료"
    exit 1
}

# 에러 시 자동 롤백
trap rollback ERR

# ---- 메인 로직 ----
log "배포 시작: 환경=${ENV}, 버전=${VERSION}"

# 1) 이미지 풀
log "이미지 다운로드 중..."
docker pull "myapp:${VERSION}"

# 2) 서비스 중지
log "서비스 중지..."
systemctl stop myapp
```

```bash
# 3) 심볼릭 링크 전환
log "버전 전환..."
ln -sfn "${APP_DIR}/releases/${VERSION}" "${APP_DIR}/current"

# 4) 서비스 시작
log "서비스 시작..."
systemctl start myapp

# 5) 헬스체크
sleep 5
if curl -sf http://localhost:8080/health > /dev/null; then
    log "배포 성공: 헬스체크 통과"
else
    log "헬스체크 실패"
    rollback
fi
```

주요 Bash 문법

```bash
# 조건문
if [ -f "/etc/nginx/nginx.conf" ]; then
    echo "Nginx 설정 파일 존재"
elif [ -d "/etc/nginx/" ]; then
    echo "Nginx 디렉터리는 있지만 설정 파일 없음"
else
    echo "Nginx 미설치"
fi

# 조건 연산자
[ -f file ]    # 파일 존재 여부
[ -d dir ]     # 디렉터리 존재 여부
[ -z "$var" ]  # 변수가 비어 있는지
[ -n "$var" ]  # 변수가 비어 있지 않은지
[ "$a" = "$b" ]  # 문자열 비교
[ "$a" -eq "$b" ] # 숫자 비교 (같음)
[ "$a" -gt "$b" ] # 숫자 비교 (초과)

# 반복문
for server in web1 web2 web3; do
    echo "Deploying to ${server}..."
    ssh "$server" "sudo systemctl restart myapp"
done

# 배열과 반복
SERVERS=("10.0.1.10" "10.0.1.11" "10.0.1.12")
for ip in "${SERVERS[@]}"; do
    ping -c 1 "$ip" && echo "${ip}: OK" || echo "${ip}: FAIL"
done

# while 루프
while ! curl -sf http://localhost:8080/health; do
```

```
    echo "서비스 대기 중..."
    sleep 2
done
echo "서비스 준비 완료"

# case 문
case "$1" in
    start)   systemctl start myapp ;;
    stop)    systemctl stop myapp ;;
    restart) systemctl restart myapp ;;
    status)  systemctl status myapp ;;
    *)       echo "Usage: $0 {start|stop|restart|status}" ;;
esac
```

파이프와 리다이렉션

```
# 파이프 (|) — 앞 명령의 출력을 뒤 명령의 입력으로
ps aux | grep nginx | grep -v grep

# 리다이렉션
command > file.txt      # 표준 출력을 파일에 (덮어쓰기)
command >> file.txt     # 표준 출력을 파일에 (추가)
command 2> error.txt    # 표준 에러를 파일에
command &> all.txt      # 표준 출력 + 에러를 모두 파일에
command < input.txt     # 파일을 표준 입력으로

# 실무 예시
# 로그에서 에러 추출하여 파일 저장
grep "ERROR" /var/log/myapp.log | sort | uniq -c | sort -rn > error_summary.txt

# 명령 결과를 변수에 저장
DISK_USAGE=$(df -h / | awk 'NR==2 {print $5}')
```

> **DevOps 팁**: 모든 자동화 스크립트는 반드시 `set -euo pipefail`로 시작하자. 에러를 무시하고 진행하면 더 큰 장애로 이어진다.

---

11. 텍스트 처리 — grep, sed, awk

로그 분석, 설정 파일 수정, 데이터 추출 등 DevOps 업무에서 텍스트 처리는 핵심 기술이다.

grep — 패턴 검색

```
# 기본 사용
grep "ERROR" /var/log/syslog          # ERROR 포함 행 검색
grep -i "error" /var/log/syslog       # 대소문자 무시
grep -r "password" /etc/               # 재귀 검색
grep -n "ERROR" app.log               # 줄 번호 표시
grep -c "ERROR" app.log               # 매칭 횟수만 출력
```

```
grep -v "DEBUG" app.log                # DEBUG 제외한 행
grep -A 3 "ERROR" app.log             # 매칭 행 + 이후 3줄
grep -B 2 "ERROR" app.log             # 매칭 행 + 이전 2줄

# 정규표현식
grep -E "^2026-04-01.*ERROR" app.log   # 확장 정규표현식
grep -P "\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}" access.log  # IP 주소 추출
```

sed — 스트림 편집기

```
# 문자열 치환
sed 's/old/new/' file.txt              # 각 행의 첫 번째만 치환
sed 's/old/new/g' file.txt             # 모든 매칭 치환
sed -i 's/old/new/g' file.txt          # 파일 직접 수정 (-i)
sed -i.bak 's/old/new/g' file.txt      # 백업 생성 후 수정

# 행 삭제
sed '/^#/d' config.txt                 # 주석 행 삭제
sed '/^$/d' config.txt                 # 빈 행 삭제

# 행 추가/삽입
sed '3a\새로운 행' file.txt             # 3번째 행 아래에 추가
sed '1i\첫 번째 행' file.txt            # 1번째 행 위에 삽입

# 실무 예시
# Nginx 포트 변경
sed -i 's/listen 80;/listen 8080;/g' /etc/nginx/nginx.conf

# 설정 파일에서 특정 값 추출
sed -n 's/.*server_name \(.*\);/\1/p' /etc/nginx/sites-available/default
```

awk — 텍스트 데이터 처리

```
# 기본 사용 (필드 분리, 기본 구분자: 공백/탭)
awk '{print $1}' file.txt              # 첫 번째 필드 출력
awk '{print $1, $3}' file.txt          # 1, 3번째 필드 출력
awk -F: '{print $1}' /etc/passwd       # 구분자를 :로 지정

# 조건 필터
awk '$3 > 80 {print $1, $3}' data.txt  # 3번째 필드가 80 초과인 행

# 내장 변수
awk '{print NR, NF, $0}' file.txt
# NR: 현재 행 번호
# NF: 현재 행의 필드 개수
# $0: 전체 행

# 실무 예시
# 디스크 사용률 80% 이상인 파티션 경고
```

```
df -h | awk 'NR>1 && int($5)>80 {print "WARNING:", $6, "사용률:", $5}'

# 로그에서 IP별 요청 횟수 상위 10개
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10

# 프로세스 메모리 사용량 상위 5개
ps aux | awk 'NR>1 {print $4, $11}' | sort -rn | head -5

# CSV 데이터 처리
awk -F, '{sum+=$3} END {print "합계:", sum}' data.csv
```

조합 활용 예시

```
# 로그 분석 파이프라인
# "지난 1시간 동안의 HTTP 500 에러를 IP별로 집계"
grep "$(date -d '1 hour ago' '+%d/%b/%Y:%H')" /var/log/nginx/access.log \
  | awk '$9 == 500 {print $1}' \
  | sort | uniq -c | sort -rn
```

---

12. 디스크와 스토리지 관리

디스크 정보 확인

```
# 디스크 사용량 확인
df -h                              # 파일시스템별 사용량 (사람이 읽기 쉽게)
df -hT                             # 파일시스템 타입 포함
du -sh /var/log/                   # 특정 디렉터리 총 사용량
du -sh /var/log/* | sort -rh | head  # 하위 디렉터리별 사용량 정렬

# 블록 디바이스 확인
lsblk                              # 블록 디바이스 트리 출력
lsblk -f                           # 파일시스템 정보 포함
fdisk -l                           # 디스크 파티션 상세 정보
blkid                              # 블록 디바이스 UUID 확인
```

디스크 파티션 및 포맷

```
# 파티션 생성 (GPT)
sudo fdisk /dev/sdb                # 대화형 파티션 관리
# 또는
sudo parted /dev/sdb mklabel gpt
sudo parted /dev/sdb mkpart primary ext4 0% 100%

# 파일시스템 생성 (포맷)
sudo mkfs.ext4 /dev/sdb1           # ext4 포맷
```

```
sudo mkfs.xfs /dev/sdb1              # xfs 포맷

# 마운트
sudo mkdir /data
sudo mount /dev/sdb1 /data

# 영구 마운트 (/etc/fstab)
echo "UUID=xxxx-xxxx /data ext4 defaults 0 2" | sudo tee -a /etc/fstab
sudo mount -a                        # fstab 기반 전체 마운트
```

LVM (Logical Volume Manager)

LVM은 물리 디스크를 논리적으로 묶어 유연하게 관리하는 기술이다.

```
┌──────────────────────────────────────────────────┐
│              논리 볼륨 (Logical Volumes)          │
│  ┌──────────┐  ┌──────────┐  ┌──────────────┐   │
│  │ lv_root  │  │ lv_data  │  │ lv_logs      │   │
│  │ 20GB     │  │ 50GB     │  │ 30GB         │   │
│  └──────────┘  └──────────┘  └──────────────┘   │
├──────────┬──────────────┬──────────────┬──────────┤
│            볼륨 그룹 (Volume Group: vg01)         │
│                     100GB                         │
├──────────────────┬──────────────────────────────┤
│     물리 볼륨     │      물리 볼륨               │
│  (PV: /dev/sda)  │   (PV: /dev/sdb)             │
│      50GB        │       50GB                   │
└──────────────────┴──────────────────────────────┘
```

```
# LVM 구성 순서
# 1) 물리 볼륨 생성
sudo pvcreate /dev/sdb /dev/sdc

# 2) 볼륨 그룹 생성
sudo vgcreate vg01 /dev/sdb /dev/sdc

# 3) 논리 볼륨 생성
sudo lvcreate -L 50G -n lv_data vg01

# 4) 포맷 및 마운트
sudo mkfs.ext4 /dev/vg01/lv_data
sudo mount /dev/vg01/lv_data /data

# 볼륨 확장 (운영 중 가능)
sudo lvextend -L +20G /dev/vg01/lv_data
sudo resize2fs /dev/vg01/lv_data     # ext4 파일시스템 확장
# 또는
sudo xfs_growfs /data                 # xfs 파일시스템 확장
```

주요 파일시스템 비교

파일시스템	최대 파일 크기	최대 볼륨 크기	특징	주요 사용처
ext4	16TB	1EB	안정적, 범용	Ubuntu 기본, 일반 서버
xfs	8EB	8EB	대용량, 고성능, 축소 불가	RHEL 기본, 대용량 데이터
btrfs	16EB	16EB	스냅샷, 압축, 서브볼륨	고급 사용, NAS
tmpfs	메모리 크기	메모리 크기	RAM 기반, 재부팅 시 소멸	`/tmp`, `/run`

> **DevOps 팁**: 클라우드 환경에서 EBS/디스크 확장 후 `resize2fs` 또는 `xfs_growfs`를 잊으면 파일시스템이 확장되지 않는다. `df -h`로 반드시 확인하자.

---

13. 로그 관리

주요 로그 파일 위치

로그 파일	내용
`/var/log/syslog` (Ubuntu)	시스템 전반 로그
`/var/log/messages` (RHEL)	시스템 전반 로그
`/var/log/auth.log` (Ubuntu)	인증/로그인 관련 로그
`/var/log/secure` (RHEL)	인증/로그인 관련 로그
`/var/log/kern.log`	커널 로그
`/var/log/nginx/access.log`	Nginx 접근 로그
`/var/log/nginx/error.log`	Nginx 에러 로그
`/var/log/cloud-init.log`	클라우드 인스턴스 초기화 로그
`/var/log/apt/history.log`	패키지 설치 이력

journalctl (systemd 로그 조회)

```
# 전체 로그
journalctl

# 특정 서비스 로그
journalctl -u nginx                  # nginx 서비스 로그
journalctl -u nginx --since "1 hour ago"  # 최근 1시간

# 시간 범위 필터
journalctl --since "2026-04-01 09:00" --until "2026-04-01 12:00"

# 부팅별 로그
journalctl -b                        # 현재 부팅
journalctl -b -1                     # 이전 부팅
```

```
# 실시간 추적
journalctl -u nginx -f              # tail -f와 유사

# 우선순위 필터
journalctl -p err                   # 에러 이상만

# 커널 로그
journalctl -k                       # dmesg와 유사

# 디스크 사용량 확인 및 정리
journalctl --disk-usage
sudo journalctl --vacuum-size=500M  # 500MB로 제한
sudo journalctl --vacuum-time=7d    # 7일 이전 삭제
```

로그 로테이션 (logrotate)

`/etc/logrotate.d/myapp` 설정 파일 예시:

```
/var/log/myapp/*.log {
    daily                   # 매일 로테이션
    rotate 14               # 14개 파일 보관
    compress                # gzip 압축
    delaycompress           # 다음 로테이션 시 압축
    missingok               # 파일 없어도 에러 아님
    notifempty              # 비어 있으면 로테이션 안 함
    create 0644 appuser appgroup  # 새 로그 파일 권한
    postrotate
        systemctl reload myapp > /dev/null 2>&1 || true
    endscript
}
```

```
# 수동 로테이션 테스트
sudo logrotate -d /etc/logrotate.d/myapp   # 디버그 (실제 실행 안 함)
sudo logrotate -f /etc/logrotate.d/myapp   # 강제 실행
```

> **DevOps 팁**: 운영 서버에서 로그 로테이션을 설정하지 않으면 디스크가 가득 차서 서비스가 중단된다. `/var/log` 사용량을 모니터링 알림에 포함하자.

---

14. 시스템 모니터링

CPU, 메모리, 디스크 모니터링

```
# CPU 정보
nproc                               # CPU 코어 수
lscpu                               # CPU 상세 정보
```

```
cat /proc/cpuinfo                    # CPU 정보 (커널)
uptime                               # 가동 시간, 로드 에버리지

# 메모리 정보
free -h                              # 메모리 사용량 (사람이 읽기 쉽게)
cat /proc/meminfo                    # 상세 메모리 정보

# 디스크 I/O
iostat -x 1                          # 디스크 I/O 통계 (1초 간격)
iotop                                # 프로세스별 디스크 I/O

# 시스템 전반
vmstat 1 5                           # 가상 메모리 통계 (1초 간격, 5회)
sar -u 1 5                           # CPU 사용률 (sysstat 패키지)
```

로드 에버리지 이해

```
$ uptime
 12:30:00 up 45 days, 3:22, 2 users, load average: 2.50, 1.80, 0.90
                                                    |      |     |
                                                    |      |     └── 15분 평균
                                                    |      └── 5분 평균
                                                    └── 1분 평균
```

CPU 코어 수	로드 에버리지	상태
4	0.5	여유 (12.5% 사용)
4	2.0	보통 (50% 사용)
4	4.0	꽉 참 (100% 사용)
4	8.0	과부하 (대기 큐 발생)

> 로드 에버리지 ÷ CPU 코어 수 = **CPU 포화도**. 1.0을 넘으면 대기 큐가 쌓이는 상태다.

top/htop 핵심 지표

```
top - 12:30:00 up 45 days, load average: 0.50, 0.80, 0.60
Tasks: 185 total,   1 running, 184 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.2 us,  1.3 sy,  0.0 ni, 92.8 id,  0.5 wa,  0.0 hi,  0.2 si
MiB Mem :  7963.3 total,   512.5 free,  4200.1 used,  3250.7 buff/cache
MiB Swap:  2048.0 total,  2040.0 free,     8.0 used.  3400.2 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM  COMMAND
 1234 www-data  20   0  320000  85000  12000 S   3.2   1.1  nginx
 5678 root      20   0  900000 250000  30000 S   2.1   3.1  dockerd
```

항목	의미
`us` (user)	사용자 프로세스 CPU 사용률
`sy` (system)	커널 프로세스 CPU 사용률
`id` (idle)	유휴 CPU (높을수록 여유)
`wa` (iowait)	I/O 대기 (높으면 디스크 병목)
`RES`	실제 물리 메모리 사용량
`VIRT`	가상 메모리 (실제보다 클 수 있음)
`buff/cache`	버퍼/캐시 (필요 시 해제 가능한 메모리)

> **DevOps 팁**: `wa` (iowait) 가 지속적으로 높으면 디스크 I/O 병목이다. EBS 볼륨 타입 변경(gp3 → io2) 또는 애플리케이션 I/O 패턴 최적화를 검토하자.

---

15. SSH와 원격 접속

SSH 기본 사용

```
# 기본 접속
ssh ubuntu@10.0.1.10

# 포트 지정
ssh -p 2222 ubuntu@10.0.1.10

# 키 파일 지정
ssh -i ~/.ssh/mykey.pem ubuntu@10.0.1.10

# 원격 명령 실행 (접속 후 바로 실행)
ssh ubuntu@10.0.1.10 "df -h && free -h"
```

SSH 키 관리

```
# 키 생성
ssh-keygen -t ed25519 -C "deploy@mycompany"
# Ed25519 권장 (RSA보다 빠르고 안전)
# 생성 결과: ~/.ssh/id_ed25519 (개인키), ~/.ssh/id_ed25519.pub (공개키)

# 원격 서버에 공개키 등록
ssh-copy-id -i ~/.ssh/id_ed25519.pub ubuntu@10.0.1.10

# 수동 등록
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

SSH 설정 파일 (~/.ssh/config)

```
# 개발 서버
Host dev
    HostName 10.0.1.10
    User ubuntu
    IdentityFile ~/.ssh/dev-key.pem
    Port 22

# 프로덕션 서버 (Bastion 경유)
Host prod-bastion
    HostName bastion.example.com
    User admin
    IdentityFile ~/.ssh/prod-key.pem

Host prod-web
    HostName 10.0.2.20
    User deploy
    ProxyJump prod-bastion
    IdentityFile ~/.ssh/prod-key.pem

# 공통 설정
Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
    AddKeysToAgent yes
```

```
# 설정 후 간편 접속
ssh dev                              # ssh ubuntu@10.0.1.10과 동일
ssh prod-web                         # Bastion 경유 자동
```

SSH 보안 강화 (`/etc/ssh/sshd_config`)

```
# 주요 보안 설정
Port 2222                          # 기본 포트(22) 변경
PermitRootLogin no                 # root 로그인 차단
PasswordAuthentication no          # 비밀번호 인증 비활성화 (키만 허용)
PubkeyAuthentication yes           # 공개키 인증 활성화
MaxAuthTries 3                     # 최대 인증 시도 횟수
LoginGraceTime 30                  # 로그인 대기 시간 (초)
AllowUsers deploy ubuntu           # 특정 사용자만 허용

# 설정 적용
sudo systemctl restart sshd
```

SCP / rsync — 파일 전송

```
# SCP: 단순 파일 복사
scp file.tar.gz ubuntu@10.0.1.10:/tmp/
scp -r /local/dir ubuntu@10.0.1.10:/remote/dir

# rsync: 증분 동기화 (대용량에 유리)
rsync -avz --progress /local/dir/ ubuntu@10.0.1.10:/remote/dir/
# -a: 아카이브 (권한, 시간 유지)
# -v: 상세 출력
# -z: 전송 중 압축
# --delete: 원본에 없는 파일 삭제 (미러링)
rsync -avz --delete /app/build/ ubuntu@10.0.1.10:/var/www/html/
```

> **DevOps 팁**: 자동화 환경에서는 SSH Agent Forwarding (`-A` 옵션) 대신 각 서버에 배포 전용 키를 배치하는 것이 보안상 안전하다. Bastion Host를 통한 ProxyJump 설정을 적극 활용하자.

---

16. cron과 작업 스케줄링

cron 기본 구조

```
┌──────────── 분 (0-59)
│ ┌──────────── 시 (0-23)
│ │ ┌──────────── 일 (1-31)
│ │ │ ┌──────────── 월 (1-12)
│ │ │ │ ┌──────────── 요일 (0-7, 0과 7은 일요일)
│ │ │ │ │
* * * * * 실행할 명령어
```

cron 표현식 예시

표현식	의미
`* * * * *`	매분 실행
`0 * * * *`	매시 정각
`0 2 * * *`	매일 새벽 2시
`0 2 * * 0`	매주 일요일 새벽 2시
`0 0 1 * *`	매월 1일 자정
`*/5 * * * *`	5분마다
`0 9-18 * * 1-5`	평일 9시~18시 매시 정각
`0 2 * * * /usr/bin/find /tmp -mtime +7 -delete`	매일 새벽 2시에 7일 이상 된 임시 파일 삭제

crontab 관리

```
# 현재 사용자의 crontab 편집
crontab -e

# crontab 목록 확인
crontab -l

# 특정 사용자의 crontab 확인 (root)
sudo crontab -u deploy -l

# crontab 예시
# ==============================================================
# 매일 새벽 3시: 데이터베이스 백업
0 3 * * * /opt/scripts/db_backup.sh >> /var/log/backup.log 2>&1

# 5분마다: 헬스체크
*/5 * * * * /opt/scripts/health_check.sh >> /var/log/health.log 2>&1

# 매주 일요일 새벽 2시: 로그 정리
0 2 * * 0 /usr/sbin/logrotate -f /etc/logrotate.conf

# 매월 1일 자정: 인증서 갱신
0 0 1 * * /usr/bin/certbot renew --quiet
# ==============================================================
```

systemd Timer (cron 대안)

systemd Timer는 cron보다 세밀한 제어가 가능하다.

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Daily Backup Timer

[Timer]
OnCalendar=*-*-* 03:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

```
# /etc/systemd/system/backup.service
[Unit]
Description=Database Backup

[Service]
Type=oneshot
ExecStart=/opt/scripts/db_backup.sh
User=backup
```

```
sudo systemctl enable --now backup.timer
systemctl list-timers                    # 활성 타이머 목록
```

비교 항목	cron	systemd Timer
의존성 관리	불가	After, Wants 설정 가능
로깅	별도 리다이렉션 필요	journalctl로 통합
정확도	분 단위	초 단위, 단조 타이머 지원
에러 처리	별도 구현 필요	Restart, OnFailure 지원
리소스 제한	불가	CPUQuota, MemoryMax 지원

> **DevOps 팁**: 새로운 프로젝트에서는 systemd Timer를 권장한다. 기존 cron 작업도 점진적으로 마이그레이션하면 로그 관리와 에러 처리가 크게 개선된다.

---

17. 네트워크 설정과 진단

네트워크 정보 확인

```
# IP 주소 확인
ip addr show                        # 전체 인터페이스
ip addr show eth0                   # 특정 인터페이스
hostname -I                         # 할당된 IP 주소만

# 라우팅 테이블
ip route show                       # 라우팅 테이블
ip route get 8.8.8.8                # 특정 목적지 경로

# DNS 확인
cat /etc/resolv.conf                # DNS 서버 설정
resolvectl status                   # systemd-resolved 상태

# 포트 확인
ss -tlnp                            # TCP 리스닝 포트
ss -ulnp                            # UDP 리스닝 포트
ss -tnp                             # 현재 TCP 연결
```

네트워크 진단 명령어

명령어	용도	사용 예시
`ping`	호스트 응답 확인	`ping -c 4 8.8.8.8`
`traceroute`	경로 추적	`traceroute google.com`
`nslookup`	DNS 조회	`nslookup example.com`

명령어	용도	사용 예시
`dig`	상세 DNS 조회	`dig example.com A +short`
`curl`	HTTP 요청 테스트	`curl -I https://example.com`
`wget`	파일 다운로드	`wget https://example.com/file.tar.gz`
`nc` (netcat)	포트 연결 테스트	`nc -zv 10.0.1.10 80`
`tcpdump`	패킷 캡처	`tcpdump -i eth0 port 80 -n`
`mtr`	ping + traceroute 통합	`mtr google.com`

방화벽 관리 (iptables / nftables / ufw)

```
# UFW (Ubuntu 간편 방화벽)
sudo ufw enable                      # 방화벽 활성화
sudo ufw allow 22/tcp                # SSH 허용
sudo ufw allow 80,443/tcp            # HTTP, HTTPS 허용
sudo ufw deny 3306/tcp               # MySQL 외부 차단
sudo ufw status verbose              # 규칙 확인

# iptables (저수준)
sudo iptables -L -n -v               # 규칙 목록
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -s 10.0.0.0/8 -j ACCEPT
sudo iptables -A INPUT -j DROP       # 기본 차단
sudo iptables-save > /etc/iptables/rules.v4   # 규칙 저장
```

네트워크 인터페이스 설정 (Netplan — Ubuntu)

```yaml
# /etc/netplan/01-netcfg.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: false
      addresses:
        - 10.0.1.10/24
      routes:
        - to: default
          via: 10.0.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

```
sudo netplan apply                    # 설정 적용
sudo netplan try                      # 테스트 (120초 후 자동 롤백)
```

> **DevOps 팁**: 클라우드 환경에서는 Security Group/NACL이 주 방화벽이지만, OS 레벨 방화벽(ufw/iptables)도 함께 설정하는 **심층 방어(Defense in Depth)** 전략을 권장한다.

---

18. 컨테이너 기반 리눅스 기술

컨테이너는 리눅스 커널의 기능 위에 구축된다. Docker와 Kubernetes를 이해하려면 그 기반 기술을 알아야 한다.

핵심 커널 기술

```
┌──────────────────────────────────────────────────────┐
│                    컨테이너 런타임                    │
│                  (Docker, containerd)                │
├──────────────────┬───────────────────────────────────┤
│                  │                                   │
│   Namespace      │           cgroup                  │
│   (격리)         │         (자원 제한)               │
│                  │                                   │
│ ┌──────────────┐ │ ┌──────────────────────────────┐  │
│ │ PID namespace │ │ │ cpu.max = 200000 100000      │ │
│ │ → 프로세스 격리│ │ │ (CPU 2코어 제한)             │ │
│ ├──────────────┤ │ ├──────────────────────────────┤ │
│ │ NET namespace │ │ │ memory.max = 512M            │ │
│ │ → 네트워크 격리│ │ │ (메모리 512MB 제한)          │ │
│ ├──────────────┤ │ ├──────────────────────────────┤ │
│ │ MNT namespace │ │ │ io.max = 100MB/s             │ │
│ │ → 파일시스템  │ │ │ (디스크 I/O 제한)            │ │
│ ├──────────────┤ │ └──────────────────────────────┘ │
│ │ UTS namespace │ │                                  │
│ │ → 호스트네임  │ │          Union FS               │
│ ├──────────────┤ │       (계층 파일시스템)          │
│ │ IPC namespace │ │                                  │
│ │ → 프로세스간통신│ │ ┌──────────────────────────┐ │
│ ├──────────────┤ │ │ 컨테이너 레이어 (쓰기 가능) │ │
│ │ USER namespace│ │ ├──────────────────────────┤ │
│ │ → 사용자 매핑 │ │ │ 앱 레이어 (읽기 전용)      │ │
│ └──────────────┘ │ ├──────────────────────────┤ │
│                  │ │ OS 레이어 (읽기 전용)      │ │
│                  │ └──────────────────────────┘ │
└──────────────────┴───────────────────────────────┘
```

Namespace 상세

Namespace	격리 대상	확인 명령어

Namespace	격리 대상	확인 명령어
PID	프로세스 ID	`ls -la /proc/self/ns/pid`
NET	네트워크 스택	`ip netns list`
MNT	마운트 포인트	`cat /proc/self/mountinfo`
UTS	호스트네임/도메인	`hostname`
IPC	프로세스 간 통신	`ipcs`
USER	사용자/그룹 ID 매핑	`cat /proc/self/uid_map`
CGROUP	cgroup 루트 디렉터리	`cat /proc/self/cgroup`

cgroup으로 리소스 제한 (v2)

```
# Docker의 --memory, --cpus 옵션이 내부적으로 cgroup을 설정
docker run --memory=512m --cpus=2 nginx

# cgroup v2 직접 확인
cat /sys/fs/cgroup/system.slice/docker-<container-id>.scope/memory.max
cat /sys/fs/cgroup/system.slice/docker-<container-id>.scope/cpu.max
```

OverlayFS (Docker 기본 스토리지 드라이버)

```
# Docker 이미지 레이어 확인
docker inspect nginx | jq '.[0].GraphDriver'

# 레이어 구조
# Lower (읽기 전용) + Upper (쓰기 가능) = Merged (통합 뷰)
# 파일 수정 시 Copy-on-Write로 Upper에 복사 후 수정
```

컨테이너 관련 리눅스 명령어

```
# 호스트에서 컨테이너 프로세스 확인
ps aux | grep containerd

# 컨테이너 내부 네임스페이스 진입
nsenter -t <PID> -n ip addr          # 네트워크 네임스페이스 진입
nsenter -t <PID> -p -r ps aux        # PID 네임스페이스 진입

# 컨테이너 리소스 사용량
docker stats                          # 실시간 리소스 모니터링
cat /proc/<PID>/cgroup               # 프로세스의 cgroup 확인
```

> **DevOps 팁**: 컨테이너 OOM(Out of Memory) Kill이 발생하면 `dmesg | grep -i oom`으로 커널 로그를 확인하고, cgroup의 `memory.max` 설정을 점검하자.

---

19. 보안 강화 (Hardening)

기본 보안 점검 목록

항목	명령어/설정	중요도
불필요한 서비스 비활성화	`systemctl disable <service>`	상
자동 보안 업데이트 활성화	`apt install unattended-upgrades`	상
root 로그인 비활성화	`PermitRootLogin no` (sshd_config)	상
SSH 키 인증만 허용	`PasswordAuthentication no`	상
방화벽 기본 차단	`ufw default deny incoming`	상
fail2ban 설치	`apt install fail2ban`	중
불필요한 SUID 파일 점검	`find / -perm -4000 -type f`	중
파일 무결성 검사	AIDE, Tripwire 등	중
커널 파라미터 강화	`/etc/sysctl.conf`	중
감사 로그 활성화	`auditd` 설치 및 규칙 설정	중

fail2ban 설정

```
# 설치
sudo apt install -y fail2ban

# 설정 파일 (/etc/fail2ban/jail.local)
sudo tee /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime  = 3600
findtime = 600
maxretry = 3

[sshd]
enabled = true
port    = ssh
logpath = /var/log/auth.log

[nginx-http-auth]
enabled = true
port    = http,https
logpath = /var/log/nginx/error.log
EOF
```

```
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd     # 차단 현황 확인
```

커널 파라미터 강화 (`/etc/sysctl.conf`)

```
# 네트워크 보안
net.ipv4.ip_forward = 0                    # IP 포워딩 비활성화 (라우터 아닌 경우)
net.ipv4.conf.all.accept_redirects = 0     # ICMP 리다이렉트 차단
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.all.rp_filter = 1            # Reverse Path Filtering
net.ipv4.tcp_syncookies = 1                # SYN Flood 방어

# 메모리 보안
kernel.randomize_va_space = 2              # ASLR 활성화
kernel.dmesg_restrict = 1                  # 일반 사용자 dmesg 제한

# 적용
sudo sysctl -p
```

SELinux / AppArmor

항목	SELinux (RHEL 계열)	AppArmor (Ubuntu 계열)
기본 배포판	RHEL, CentOS, Rocky	Ubuntu, Debian
모드	Enforcing, Permissive, Off	Enforce, Complain, Off
상태 확인	`getenforce`, `sestatus`	`sudo apparmor_status`
정책 방식	라벨 기반 (Label)	경로 기반 (Path)
학습 용이성	복잡	상대적으로 간단

```
# SELinux
getenforce                          # 현재 모드 확인
sudo setenforce 0                   # 임시로 Permissive 전환
# /etc/selinux/config 에서 영구 변경

# AppArmor
sudo apparmor_status                # 프로필 상태 확인
sudo aa-complain /etc/apparmor.d/usr.sbin.nginx  # Complain 모드
sudo aa-enforce /etc/apparmor.d/usr.sbin.nginx   # Enforce 모드
```

> **DevOps 팁**: 컨테이너 환경에서는 SELinux/AppArmor에 더해 **seccomp** 프로필로 시스템 콜을 제한하고, `--read-only` 파일시스템과 `--no-new-privileges` 옵션을 활용하여 보안을 강화한다.

---

20. 트러블슈팅 가이드

장애 대응 기본 흐름

```
장애 발생
    |
    v
[1단계] 증상 파악
    |   → 어떤 서비스? 에러 메시지? 언제부터?
    v
[2단계] 로그 확인
    |   → journalctl -u service -f
    |   → /var/log/ 확인
    v
[3단계] 리소스 확인
    |   → CPU: top, htop
    |   → 메모리: free -h
    |   → 디스크: df -h
    |   → 네트워크: ss -tlnp
    v
[4단계] 프로세스 확인
    |   → ps aux | grep service
    |   → systemctl status service
    v
[5단계] 원인 분석 및 조치
    |   → 설정 수정, 재시작, 스케일업 등
    v
[6단계] 검증 및 문서화
    → 정상 동작 확인, 사후 분석 문서 작성
```

상황별 트러블슈팅 명령어

디스크 공간 부족

```
# 어디서 공간을 차지하는지 확인
df -h                                # 파일시스템별 사용량
du -sh /* 2>/dev/null | sort -rh | head -10  # 최상위 디렉터리별
du -sh /var/log/* | sort -rh | head  # 로그 디렉터리 상세

# 대용량 파일 찾기
find / -type f -size +100M -exec ls -lh {} \; 2>/dev/null

# 삭제된 파일이 여전히 공간 차지 (프로세스가 잡고 있는 경우)
lsof +L1                            # 삭제되었지만 열려 있는 파일

# 정리
sudo journalctl --vacuum-size=200M   # systemd 로그 정리
sudo apt autoremove                  # 불필요한 패키지 제거
docker system prune -a               # Docker 미사용 리소스 정리
```

서비스 시작 실패

```
# 상태 및 로그 확인
systemctl status myapp
journalctl -u myapp --no-pager -n 50

# 설정 파일 문법 검사
nginx -t                              # Nginx 설정 테스트
apachectl configtest                  # Apache 설정 테스트

# 포트 충돌 확인
ss -tlnp | grep :8080                 # 해당 포트를 쓰는 프로세스

# 권한 문제 확인
namei -l /opt/myapp/server.js         # 경로 전체의 권한 확인
```

높은 CPU 사용률

```
# CPU 많이 사용하는 프로세스 찾기
top -o %CPU                           # CPU 기준 정렬
ps aux --sort=-%cpu | head -10        # 상위 10개

# 특정 프로세스 상세 분석
strace -p <PID> -c                    # 시스템 콜 통계
perf top                              # 실시간 성능 프로파일링
```

메모리 부족

```
# 메모리 상태 확인
free -h
cat /proc/meminfo | grep -i "mem\|swap"

# 메모리 많이 사용하는 프로세스
ps aux --sort=-%mem | head -10

# OOM Killer 로그 확인
dmesg | grep -i "oom\|out of memory"
journalctl -k | grep -i "oom"

# 캐시 수동 해제 (긴급 시)
sudo sync && echo 3 | sudo tee /proc/sys/vm/drop_caches
```

네트워크 연결 문제

```
# DNS 확인
dig example.com +short
nslookup example.com

# 포트 연결 테스트
nc -zv 10.0.1.10 80                    # TCP 연결 테스트
curl -v https://example.com            # HTTP 상세 요청

# 라우팅 확인
ip route show
traceroute 10.0.1.10

# 패킷 캡처 (상세 분석)
sudo tcpdump -i eth0 port 80 -n -c 20
```

유용한 종합 명령어

명령어	용도
`dmesg -T`	커널 메시지 (타임스탬프 포함)
`last`	최근 로그인 기록
`lastb`	실패한 로그인 기록
`w`	현재 로그인한 사용자와 실행 중인 명령
`lsof -i :80`	80 포트 사용 프로세스
`strace -p PID`	프로세스의 시스템 콜 추적
`vmstat 1`	CPU, 메모리, I/O 실시간 통계
`sar -n DEV 1`	네트워크 인터페이스 트래픽 통계

> **DevOps 팁**: 트러블슈팅의 핵심은 **체계적 접근**이다. "아마 이거겠지"로 설정을 바꾸기보다, 로그와 지표를 확인하여 근거를 기반으로 원인을 좁혀 나가자.

---

21. 참고 자료

공식 문서

- Linux man pages — 리눅스 명령어 공식 매뉴얼
- The Linux Documentation Project — 리눅스 가이드/HOWTO 문서
- systemd documentation — systemd 공식 문서
- Ubuntu Server Guide — Ubuntu 서버 공식 가이드
- Red Hat System Administration — RHEL 시스템 관리 문서

학습 자료

- Linux Journey — 초보자를 위한 단계별 리눅스 학습

- OverTheWire: Bandit — 리눅스 명령어 워게임
- Bash Guide (Greg's Wiki) — Bash 스크립팅 심화 가이드
- Linux Performance (Brendan Gregg) — 리눅스 성능 분석 바이블

도구

- Explain Shell — 리눅스 명령어 구문 분석기
- crontab.guru — cron 표현식 해석기
- ShellCheck — 쉘 스크립트 정적 분석 도구

> **문서 작성일**: 2026-04-01
> **참고**: 리눅스 생태계는 빠르게 발전합니다. 주요 변경사항은 각 배포판 및 프로젝트의 공식 문서를 확인하시기 바랍니다.